University of Kelaniya

Faculty of Computing Technology

Internet of Things - CTEC 32023

Mini-Project

Off-Grid Smart Agriculture Node

Final Project Report

Group Members:

- **CT/2021/014 – W.A.K.H.Wijesingha**
- **CT/2021/015 – N.P. Wijewardhana**
- **CT/2021/045 – H.G.G.I.Ranawee**

Contents

1. Introduction 3
2.Project Objectives 3
3.Design Process 3
Step 1 – Planning 3
Step 2 – Hardware Design 3
Step 3 – Software Development 4
Step 4 – Testing 4
04.Challenges Faced and Solutions Implemented 4
5. Screenshots of the Working Prototype 5
Figure 1 – Circuit Connections 5
Figure 2 – Complete Hardware Prototype 5
Figure 3 – Blynk Dashboard 7
Figure 4 – Mobile Application Interface 8
6. Source Code with Comments 9
7. Instructions to Replicate the Project 15
8. Conclusion 16

1. Introduction

Agriculture is one of the most important industries in Sri Lanka. Farmers need to monitor soil moisture, temperature, humidity, and sunlight to grow healthy crops. Many farms are located in areas without reliable electricity or internet. This project introduces an **Off-Grid Smart Agriculture Node**, which uses solar power and IOT technology to monitor environmental conditions and send data to the cloud for remote monitoring.

2.Project Objectives

The main objective of this project was to design and develop an **Off-Grid Smart Agriculture Node** capable of monitoring environmental conditions in agricultural fields without relying on the electrical grid. The system collects soil moisture, temperature, humidity, and light intensity data and transmits the information to a cloud platform using GSM communication. The project is powered by a solar panel and rechargeable battery, making it suitable for remote farming areas.

Specific Objectives

- Monitor soil moisture levels.
- Measure temperature and humidity.
- Measure light intensity.
- Upload sensor data to the cloud platform.
- Enable remote monitoring using a mobile device.
- Operate using renewable solar energy.

3.Design Process

The development of the project followed several stages.

Step 1 – Planning

The project requirements were identified, and suitable hardware and software components were selected.

Step 2 – Hardware Design

The following hardware components were connected to the ESP32 microcontroller:

- ESP32 NodeMCU
- DHT22 Sensor
- Soil Moisture Sensor
- LDR Sensor
- SIM900A GSM Module
- Solar Panel
- TP4056 Charging Module

- Rechargeable Battery
- Boost Converter

The ESP32 reads the sensor values and communicates with the GSM module to upload data to the cloud.

Step 3 – Software Development

The system was programmed using **Arduino IDE**. The TinyGSM, Blynk, and DHT libraries were used to collect sensor data and send it to the cloud platform. The ESP32 enters deep sleep mode after transmitting data to reduce power consumption.

Step 4 – Testing

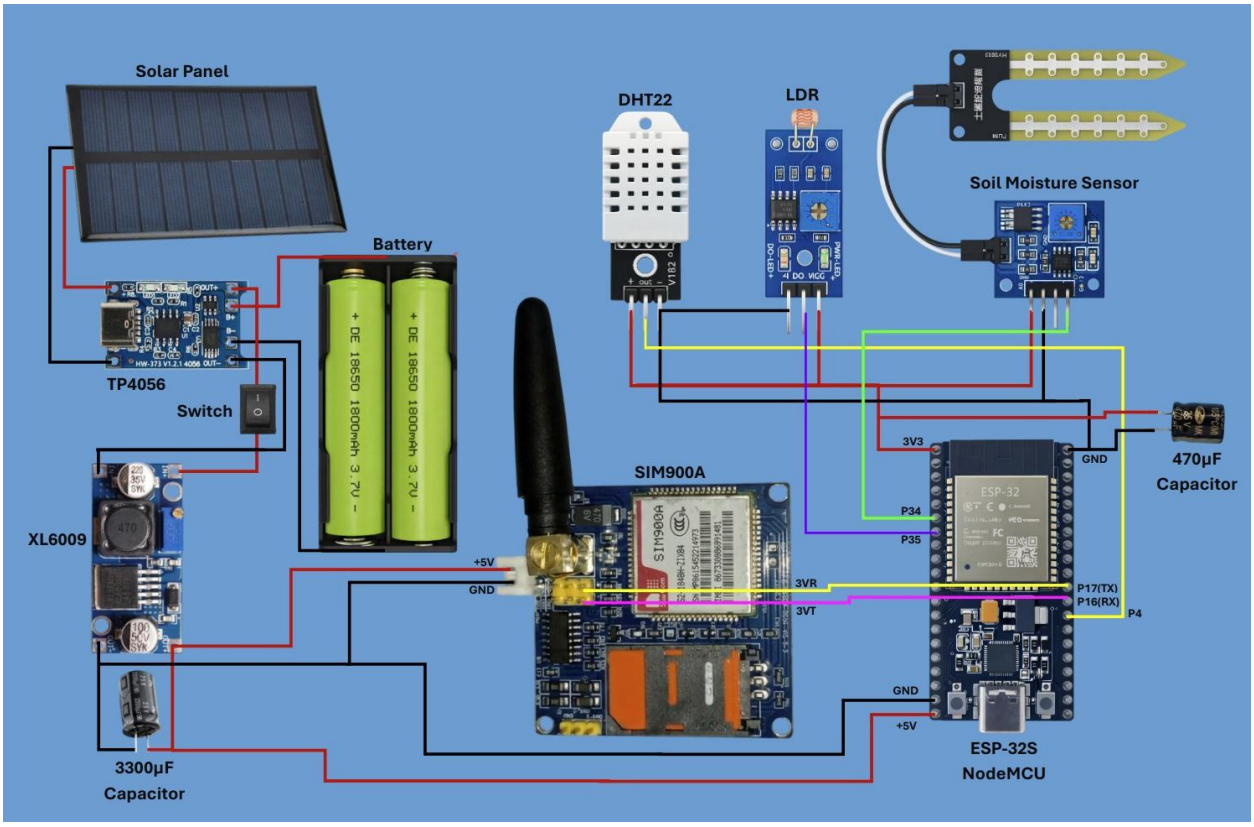
Each sensor and communication module was tested individually before integrating the complete system.

04.Challenges Faced and Solutions Implemented

Challenge	Solution
GSM module restarted because of high current consumption	Added a 3300 µF capacitor across the 5 V power supply to stabilize the voltage.
Sensor readings fluctuated because of electrical noise	Added a 470 µF capacitor on the 3.3 V power line.
GSM communication errors due to improper grounding	Connected all components to a common ground (GND).
High battery consumption during continuous operation	Implemented ESP32 deep sleep mode with a 5-minute wake-up interval.
Charging module became hot during charging	Attached a small aluminum heatsink to improve cooling.

5. Screenshots of the Working Prototype

Figure 1 – Circuit Connections



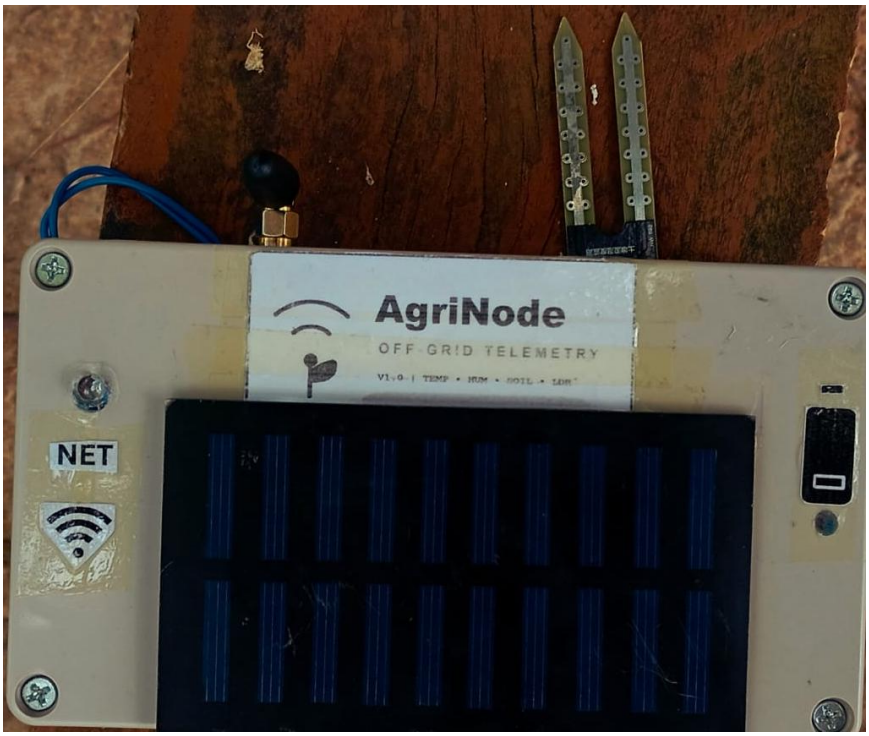
(Insert the circuit diagram created using EasyEDA or another circuit design tool.)

Figure 2 – Complete Hardware Prototype



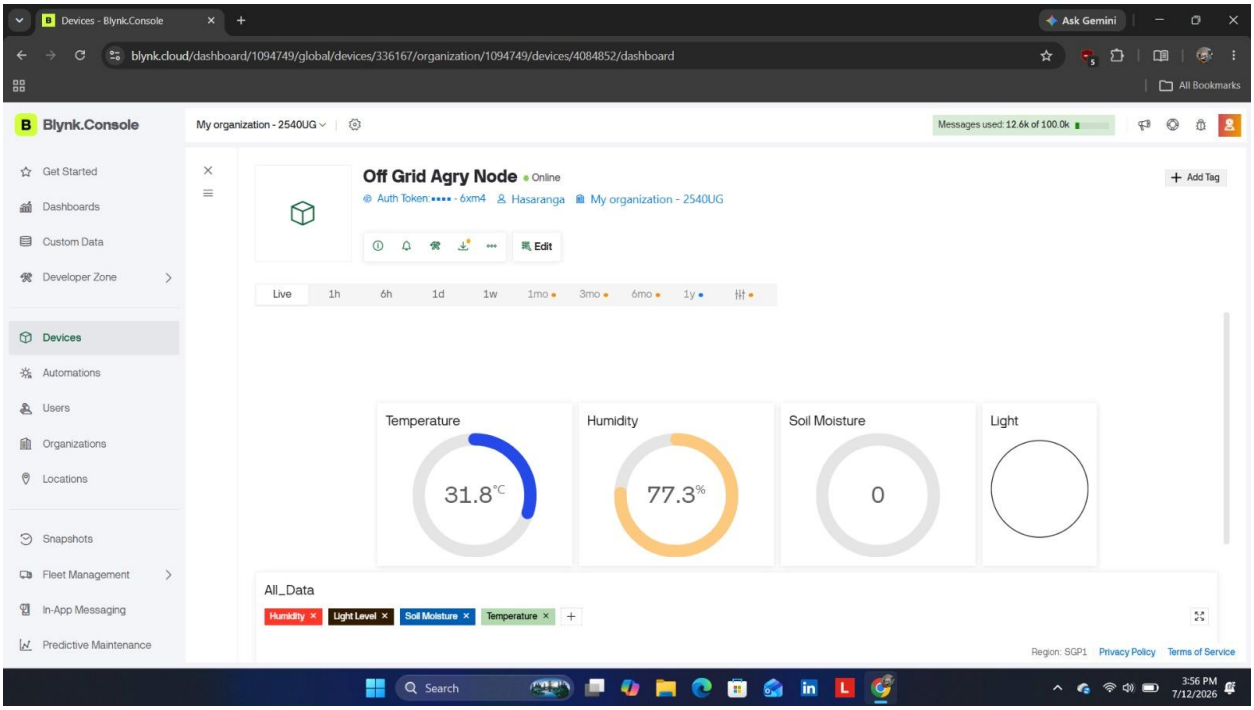

DHT22
LDR
Soil
Moisture


AgriNode
NET



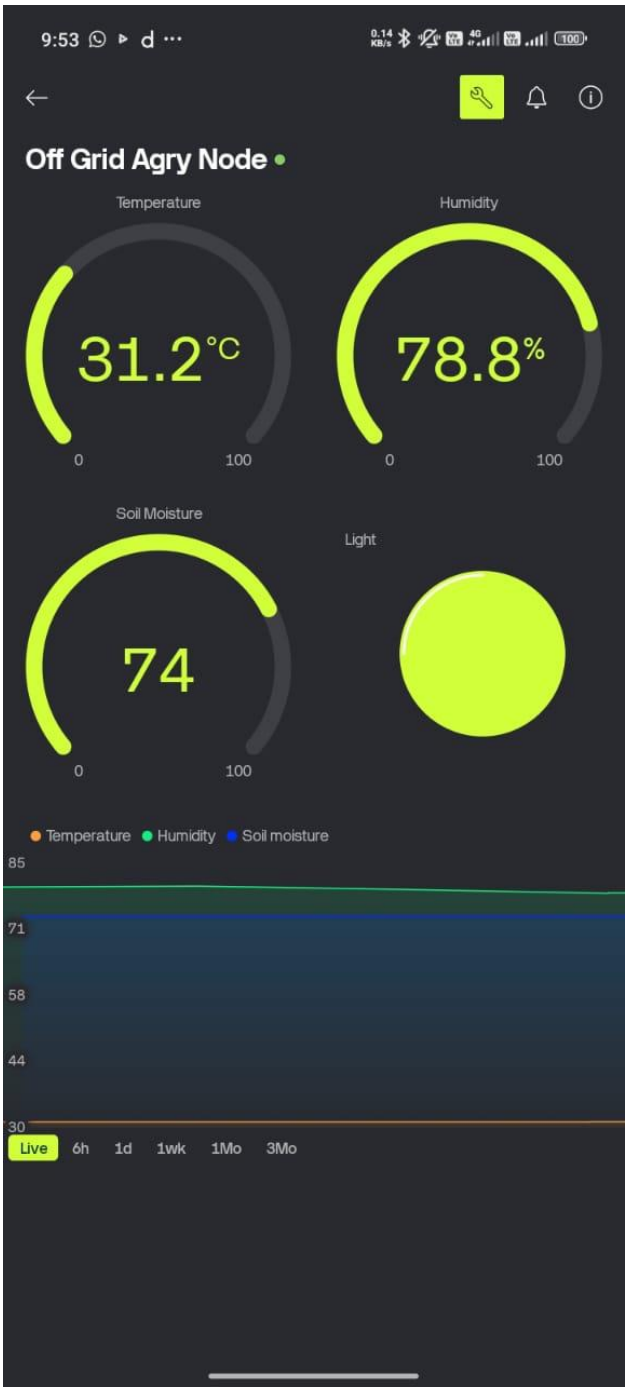
(Insert a photo of the assembled ESP32, sensors, GSM module, battery, and solar panel.)

Figure 3 – Blynk Dashboard



(Insert a screenshot showing temperature, humidity, soil moisture, and light intensity values.)

Figure 4 – Mobile Application Interface



(Insert screenshots of the Blynk mobile app displaying the live sensor readings.)

6. Source Code with Comments

```
/*
================================================================================
* Project: Off-Grid Agri Node
* Description: A solar-powered, 18650 battery-backed IoT agricultural sensor node.
* Uses an ESP32 and a SIM900A module to push environmental telemetry
* (Temperature, Humidity, Soil Moisture, Light) to a Blynk cloud dashboard
* via a 2G GPRS connection.
* * Key Features:
* - Advanced Power Sequencing: Utilizes ESP32 RTC memory and Deep Sleep to isolate
* power delivery, preventing brownouts during the SIM900A's 2-Amp network search spike.
* - Power Optimization: Disables internal Wi-Fi/Bluetooth and dynamically switches
* sensor power via a GPIO pin to maximize battery life.
* - Visual Diagnostics: Uses the onboard LED to flash specific hardware error codes.
*
================================================================================ */
// --- BLYNK CLOUD CREDENTIALS ---
#define BLYNK_TEMPLATE_ID "TMPL6_cIRv-mg"
#define BLYNK_TEMPLATE_NAME "Off Grid Agry Node"
#define BLYNK_AUTH_TOKEN "rGYOZ-cxFLtzIlOXYxYEJZCDCZDb6xm4"
// --- HIGH LATENCY 2G NETWORK OVERRIDES ---
// Increases timeout thresholds to prevent Blynk from disconnecting over slow 2G networks
#define BLYNK_HEARTBEAT 60
#define BLYNK_TIMEOUT_MS 10000
#define TINY_GSM_MODEM_SIM800
#include <TinyGsmClient.h>
#include <BlynkSimpleTinyGSM.h>
#include <DHT.h>
#include <WiFi.h>
```

```
// --- CELLULAR APN SETTINGS ---
const char apn[]      = "hutch3g";
const char gprsUser[] = "";
const char gprsPass[] = "";
// --- HARDWARE PIN ASSIGNMENTS ---
#define SerialAT Serial2
#define SIM_RX   16
#define SIM_TX   17
#define SIM_BAUD 9600
#define DHTPIN
4
#define DHTTYPE       DHT22
#define SOIL_PIN      34
#define LDR_PIN       35
// Power Control & Diagnostics
#define SENSOR_POWER_PIN 32  // Controls a transistor/relay to cut power to sensors when not in use
#define ONBOARD_LED      2   // Used for connection success and visual error codes
// --- RTC MEMORY ---
// Variables stored in RTC memory survive Deep Sleep resets.
// Used to track if the system is booting from a hard power switch or waking from a nap.
RTC_DATA_ATTR bool isFirstBoot = true;
// --- TIMING CONSTANTS ---
#define TIME_TO_SLEEP_BOOT 30
// 30-second hardware isolation nap for modem boot
#define uS_TO_S_FACTOR 1000000ULL    // Conversion factor for micro seconds to seconds
// --- COMPONENT INITIALIZATION ---
DHT dht(DHTPIN, DHTTYPE);
TinyGsm modem(SerialAT);
BlynkTimer timer;
/**
```

```
* @brief Visual Diagnostic Tool
* Freezes the main loop and flashes the onboard LED to indicate hardware faults without needing a Serial Monitor.
* 1 Blink  = Cannot communicate with SIM900A
* 2 Blinks = Cannot find Cellular Network Tower
* 3 Blinks = GPRS Data Pipe rejected (Check APN or Data Balance)
* 4 Blinks = Blynk Server Login Timeout
*/
void flashError(int blinks) {
while(true) {
for(int i = 0; i < blinks; i++) {
digitalWrite(ONBOARD_LED, HIGH);
delay(250);
digitalWrite(ONBOARD_LED, LOW);
      delay(250);
    }
    delay(1500); // Pause before repeating the error sequence
  }
}

/**
 * @brief Safely maps raw ADC values to a 0-100 percentage.
 * Prevents UI layout breaks on the dashboard by constraining values that drift out of bounds.
 */
int mapConstrained(int value, int fromLow, int fromHigh, int toLow, int toHigh) {
  int result = map(value, fromLow, fromHigh, toLow, toHigh);
  return constrain(result, toLow, toHigh);
}

/**
```

```
 * @brief Reads all environmental sensors and pushes the payload to the Blynk cloud.
 */
void pushDataToCloud() {
  float humidity = dht.readHumidity();
  float temperature = dht.readTemperature();
  int soilRaw = analogRead(SOIL_PIN);
  int ldrRaw = analogRead(LDR_PIN);

  // Convert raw 12-bit ADC values (0-4095) to human-readable percentages
  int soilPercent = mapConstrained(soilRaw, 3800, 800, 0, 100);
  int lightPercent = mapConstrained(ldrRaw, 4095, 100, 0, 100);

  // Safety Check: Only transmit if the DHT22 provides valid numerical data
  if (!isnan(humidity) && !isnan(temperature)) {
    Blynk.virtualWrite(V0, temperature);
    Blynk.virtualWrite(V1, humidity);
    Blynk.virtualWrite(V2, soilPercent);
    Blynk.virtualWrite(V3, lightPercent);
  }
}

void setup() {
  // --- STAGE 1: MAXIMUM POWER SAVING ---
  // Immediately disable unused wireless radios to free up ~150mA for the power supply
  WiFi.mode(WIFI_OFF);
  btStop();

  pinMode(ONBOARD_LED, OUTPUT);
  digitalWrite(ONBOARD_LED, LOW);
```

```
  // Isolate hardware to prevent "Phantom Powering" where the ESP32 powers the SIM900A through data lines
  pinMode(SIM_RX, INPUT);
  pinMode(SIM_TX, INPUT);

  // Ensure peripheral sensors remain completely off during the heavy cellular boot phase
  pinMode(SENSOR_POWER_PIN, OUTPUT);
  digitalWrite(SENSOR_POWER_PIN, LOW);

  // --- STAGE 2: HARDWARE ISOLATION NAP (COLD BOOT ONLY) ---
  // The SIM900A requires a 2-Amp surge to connect to the cell tower. To prevent battery voltage
  // from collapsing (Brownout), the ESP32 goes into a dead sleep for 30 seconds, giving the modem
  // 100% of the available system current.
  if (isFirstBoot) {
    isFirstBoot = false; // Update RTC memory so it bypasses this stage on future wake-ups
    esp_sleep_enable_timer_wakeup(TIME_TO_SLEEP_BOOT * uS_TO_S_FACTOR);
    esp_deep_sleep_start();
  }

  // --- STAGE 3: MODEM WAKE-UP & HANDSHAKE ---
  SerialAT.begin(SIM_BAUD, SERIAL_8N1, SIM_RX, SIM_TX);
  delay(2000);

  // Ping the SIM900A UART to wake it from its own idle sleep state
  SerialAT.println("AT");
  delay(500);
  TinyGsmAutoBaud(SerialAT, 9600, 9600);

  // --- STAGE 4: CONNECTION PIPELINE ---
  if (!modem.init()) {
```

```
  flashError(1);
 }
 if (!modem.waitForNetwork(60000L)) {
  flashError(2);
 }
 if (!modem.gprsConnect(apn, gprsUser, gprsPass)) {
  flashError(3);
 }

 Blynk.config(modem, BLYNK_AUTH_TOKEN, "blynk.cloud", 8080);

 if (!Blynk.connect(20000)) {
  flashError(4);
 }

 // --- STAGE 5: SYSTEM SUCCESS & DATA CAPTURE ---
 // Connection verified. Turn on indicator LED and power up the sensor array.
 digitalWrite(ONBOARD_LED, HIGH);
 digitalWrite(SENSOR_POWER_PIN, HIGH);
 delay(2000); // Allow DHT22 internal capacitor to charge before reading

 dht.begin();
 timer.setInterval(10000L, pushDataToCloud);
}

void loop() {
 Blynk.run();
 timer.run();
}
```

7. Instructions to Replicate the Project

Follow these steps to rebuild the project:

1. Gather the required hardware components:
    - ESP32
    - DHT22 Sensor
    - Soil Moisture Sensor
    - LDR Sensor
    - SIM900A GSM Module
    - Solar Panel
    - TP4056 Charging Module
    - 18650 Rechargeable Battery
    - Boost Converter
2. Connect all components according to the circuit diagram.
3. Install **Arduino IDE**.
4. Install the required libraries:
    - TinyGSM
    - Blynk
    - DHT Sensor Library
5. Update the source code with:
    - Blynk Authentication Token
    - APN settings for the SIM card
6. Upload the program to the ESP32.
7. Insert an activated SIM card into the GSM module.
8. Power the system using the solar panel and battery.
9. Open the Blynk mobile application to monitor the real-time sensor values.
10. Verify that temperature, humidity, soil moisture, and light intensity data are uploaded successfully to the cloud platform

8. Conclusion

The **Off-Grid Smart Agriculture Node** is an effective IoT-based farming solution. It enables farmers to monitor environmental conditions remotely while operating entirely on solar energy. The project reduces manual work, improves water management, and supports smart agriculture in remote locations without access to grid electricity.